

BUILD WEEK 3 // ESERCIZIO 6

# Estrarre un Eseguibile da un PCAP

Analisi di traffico catturato e recupero  
di un file eseguibile da una cattura di rete  
tramite Wireshark

CyberOps Workstation | Wireshark | Follow TCP Stream

|               |                                                  |
|---------------|--------------------------------------------------|
| ESERCITAZIONE | Esercizio 6 -- Estrarre un Eseguibile da un PCAP |
|---------------|--------------------------------------------------|

|          |                                           |
|----------|-------------------------------------------|
| AMBIENTE | CyberOps Workstation (Linux) -- Wireshark |
|----------|-------------------------------------------|

|                 |                                   |
|-----------------|-----------------------------------|
| FILE ANALIZZATO | nimda.download.pcap (371462 byte) |
|-----------------|-----------------------------------|

|        |       |
|--------|-------|
| AUTORE | _____ |
|--------|-------|

|      |       |
|------|-------|
| DATA | _____ |
|------|-------|



00

## Indice

L'indice seguente anticipa l'intera sequenza di lavoro: dall'analisi a livello di pacchetto della transazione di rete, fino all'estrazione e all'identificazione dell'eseguibile contenuto nella cattura.

### 1. Obiettivo

### 2. Configurazione dell'ambiente

### 3. Parte 1: Analisi di log e catture pre-catturate

- 3.1 Apertura del file e handshake TCP
- 3.2 Identificazione della richiesta HTTP GET
- 3.3 Follow TCP Stream e analisi del contenuto
- 3.4 Risposta alle domande di analisi

### 4. Parte 2: Estrarre il file scaricato dal PCAP

- 4.1 Export Objects HTTP
- 4.2 Perché W32.Nimda.Amm.exe è l'unico file nella cattura
- 4.3 Salvataggio del file
- 4.4 Identificazione del tipo di file
- 4.5 Quale sarebbe un probabile passo successivo

### 5. Riepilogo dei comandi

### 6. Conclusioni

## 01 Obiettivo

In questo laboratorio ho lavorato sul file `nimda.download.pcap`, una cattura di rete realizzata in un esercizio precedente che contiene i pacchetti relativi al download di un file eseguibile. L'obiettivo è duplice: nella prima parte ho analizzato a livello di pacchetto come avviene una transazione di rete, individuando l'handshake TCP, la richiesta HTTP e il flusso completo della trasmissione; nella seconda parte ho estratto l'eseguibile direttamente dal PCAP e ne ho verificato la natura.

Capire come avvengono le transazioni a livello di pacchetto è importante quanto leggere i log: un log mi dice che qualcosa è successo, mentre il pacchetto mi mostra esattamente cosa è stato trasferito, byte per byte. Questo è il principio su cui si basa il recupero di un artefatto da una cattura di rete.

## 02 Configurazione dell'ambiente

Ho usato la macchina virtuale CyberOps Workstation. Il file di cattura si trova nella directory dedicata alle risorse di laboratorio.

| Voce                 | Valore                                             |
|----------------------|----------------------------------------------------|
| Sistema              | CyberOps Workstation (Linux)                       |
| Strumento di analisi | Wireshark                                          |
| File analizzato      | <code>nimda.download.pcap</code> (371462 byte)     |
| Percorso del PCAP    | <code>/home/analyst/lab.support.files/pcaps</code> |
| Client del download  | 209.165.200.235 (wget)                             |
| Server del download  | 209.165.202.133:6666 (nginx/1.12.0)                |
| File trasferito      | W32.Nimda.Amm.exe (345088 byte)                    |

Ho raggiunto la cartella di lavoro e verificato la presenza del file con un elenco dettagliato, poi ho aperto la cattura in Wireshark lanciandolo in background.

```
[analyst@secOps home]$ cd analyst/  
[analyst@secOps ~]$ cd lab.support.files/  
[analyst@secOps lab.support.files]$ cd pcaps/  
[analyst@secOps pcaps]$ ls -la  
total 4036  
-rw-r--r-- 1 analyst analyst 371462 Mar 21 2018 nimda.download.pcap  
-rw-r--r-- 1 analyst analyst 3750153 Mar 21 2018 wannacry_download_pcap.pcap  
[analyst@secOps pcaps]$ wireshark nimda.download.pcap &
```

### Nota

All'avvio Wireshark ha stampato un avviso `qt.multimedia.symbolsresolver: Couldn't load pipewire-0.3 library`. È un messaggio relativo alle librerie audio di Qt e non ha alcun impatto sull'analisi della cattura: l'ho ignorato.

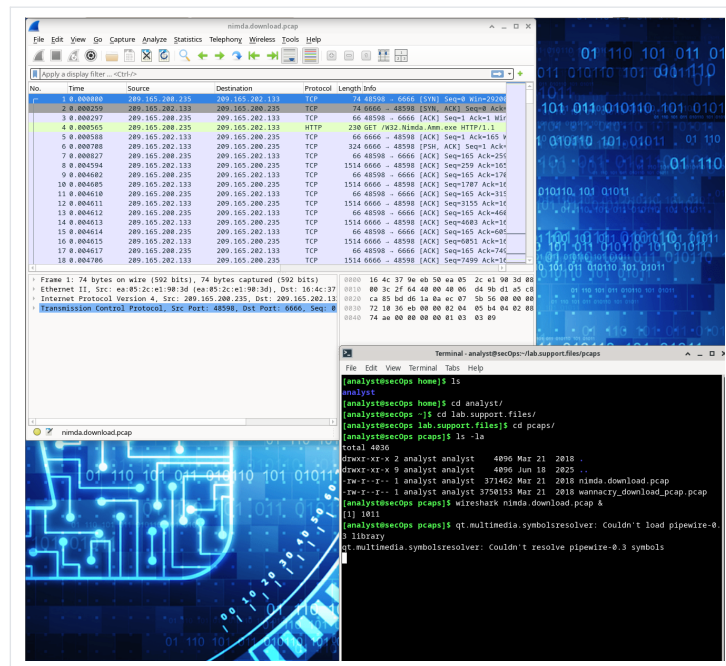


Figura 1: ambiente di lavoro. A sinistra Wireshark con la packet list di nimda.download.pcap, a destra il terminale con la navigazione fino a lab.support.files/pcaps e l'avvio della cattura.

## 03

## Parte 1: Analisi di log e catture pre-catturate

- 3.1 Apertura del file e handshake TCP

**Cosa ho osservato**

I primi tre pacchetti della cattura sono l'handshake TCP a tre vie tra il client 209.165.200.235 e il server 209.165.202.133 sulla porta di destinazione 6666.

| Pacchetto | Direzione           | Flag     | Significato                                    |
|-----------|---------------------|----------|------------------------------------------------|
| 1         | client verso server | SYN      | il client chiede di aprire la connessione      |
| 2         | server verso client | SYN, ACK | il server accetta e risponde                   |
| 3         | client verso server | ACK      | il client conferma, la connessione è stabilita |

**Perché è importante**

Una connessione HTTP viaggia sempre dentro una connessione TCP, e TCP prima di trasportare qualsiasi dato deve completare questo scambio in tre fasi. Vedere l'handshake completo mi conferma che la trasmissione successiva è una sessione TCP regolare e ben formata, non traffico spurio o frammenti isolati.

**Cosa significa**

Dal pacchetto 4 in poi la connessione è pronta a trasportare dati applicativi, e infatti è lì che compare la richiesta HTTP.

- 3.2 Identificazione della richiesta HTTP GET

Ho selezionato il quarto pacchetto, l'unico marcato come protocollo HTTP nella lista, ed espanso la sezione Hypertext Transfer Protocol.

```
GET /W32.Nimda.Amm.exe HTTP/1.1
User-Agent: Wget/1.19.1 (linux-gnu)
Accept: */*
Accept-Encoding: identity
Host: 209.165.202.133:6666
Connection: Keep-Alive
```

**Cosa ho verificato**

Il download è stato fatto con una richiesta HTTP di tipo GET verso il percorso /W32.Nimda.Amm.exe. Wireshark indica anche che la risposta del server arriva nel frame 309 e che l'URI completo richiesto è http://209.165.202.133:6666/W32.Nimda.Amm.exe.

### Perche e un dettaglio interessante

Il campo user-Agent non e un browser ma wget/1.19.1 (linux-gnu). Significa che il file non e stato scaricato da una navigazione interattiva, ma da uno strumento da riga di comando, comportamento tipico di uno script automatizzato o di un download deliberato di un payload.

### Cosa significa

A questo punto so quale file e stato richiesto, da chi e con quale strumento. Mi manca di capire cosa contiene davvero quel file, e per questo passo al flusso completo.

The image shows a Wireshark packet capture of an HTTP GET request. The packet list on the left shows packet 4 selected. The packet details pane on the right shows the structure of the packet: Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Hypertext Transfer Protocol. The Hypertext Transfer Protocol section shows the request line: GET /W32.Nimda.Amm.exe HTTP/1.1. The User-Agent field is set to 'wget/1.19.1 (linux-gnu)'. The Host field is '209.165.202.133:6666'. The Connection field is 'Keep-Alive'. The packet bytes pane at the bottom shows the raw data of the packet.

Figura 2: pacchetto 4 con la sezione Hypertext Transfer Protocol espansa. Si legge la richiesta GET, lo User-Agent Wget e l'URI completo dell'eseguibile.

### 3.3 Follow TCP Stream e analisi del contenuto

Poiche HTTP viaggia su TCP, ho usato la funzione Follow TCP Stream di Wireshark partendo dal primo pacchetto SYN. Questa funzione ricostruisce l'intera transazione TCP, mettendo in fila tutti i byte scambiati tra client e server come se fossero un unico testo continuo.

La risposta del server e la seguente:

```
HTTP/1.1 200 OK
Server: nginx/1.12.0
Date: Tue, 02 May 2017 14:26:50 GMT
Content-Type: application/octet-stream
Content-Length: 345088
Last-Modified: Fri, 14 Apr 2017 19:17:25 GMT
Connection: keep-alive
ETag: "58f12045-54400"
Accept-Ranges: bytes
```

Il Content-Type: application/octet-stream indica un flusso binario generico, cioe un file da scaricare e non da visualizzare nel browser, e Content-Length: 345088 mi dice gia la dimensione esatta del file in arrivo.

```

.....s.....h.....@.....(
.....P.....h.....4.....PZ.....
.....j.....h.....0.....h.....P.....I.D.I._.A.P.P.I.C.O.N...M.U.I.....
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!-- Copyright (c) Microsoft Corporation -->
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
  <assemblyIdentity
    version="5.1.0.0"
    processorArchitecture="amd64"
    name="Microsoft.Windows.FileSystem.CMD"
    type="win32"
  />
  <description>Windows Command Processor</description>
  <trustInfo xmlns="urn:schemas-microsoft-com:asm.v3">
    <security>
      <requestedPrivileges>
        <requestedExecutionLevel
          level="asInvoker"
          uiAccess="false"
        />
      </requestedPrivileges>
    </security>
  </trustInfo>
  <application xmlns="urn:schemas-microsoft-com:asm.v3">
    <windowsSettings>
      <dpiAware xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSettings">true</dpiAware>
    </windowsSettings>
  </application>
</assembly>

```

Figura 3: inizio del Follow TCP Stream. In alto la richiesta del client e la risposta HTTP 200 OK del server, poi l'avvio del contenuto binario con la firma MZ e il DOS stub.

### ● 3.4 Risposta alle domande di analisi

I simboli mostrati nella finestra Follow TCP Stream sono rumore di connessione o dati?

#### Cosa sono

Non sono rumore di connessione, sono i dati veri e propri. Quel mare di puntini e il contenuto binario del file eseguibile, mostrato in rappresentazione ASCII. I byte che corrispondono a caratteri stampabili appaiono come testo leggibile, mentre tutti gli altri byte (la maggior parte di un file binario) vengono resi come punti, perché non hanno un carattere ASCII visualizzabile. Quindi sto guardando l'eseguibile stesso che transita sulla rete.

Ci sono parole leggibili sparse tra i simboli. Perché sono lì?

#### Perché sono lì

Le parti leggibili sono le porzioni di testo che ogni file PE di Windows contiene al suo interno. In particolare ho riconosciuto:

- MZ all'inizio del file, la firma del formato eseguibile PE, seguita dal DOS stub con la frase `This program cannot be run in DOS mode.`;
- la firma Rich, impronta lasciata dal linker Microsoft;
- la sigla PE e i nomi delle sezioni del file: `.text`, `.rdata`, `.data`, `.pdata`, `.rsrc`, `.reloc`;
- i nomi delle librerie importate: `KERNEL32.dll`, `ntdll.dll`, `msvcrt.dll`, `ADVAPI32.dll`, `SHELL32.dll`, `MPR.dll`, `USER32.dll`, `WINBRAND.dll`, `api-ms-win-core-processthreads-l1-1-0.DLL`;
- un manifest XML completo incorporato nelle risorse del file.

### Cosa significa

Queste stringhe non sono casuali, sono la struttura interna di un eseguibile Windows che diventa leggibile perche composta di testo. Sono proprio queste parole a permettermi di identificare il file.

### Domanda Sfida: qual e realmente l'eseguibile, nonostante il nome W32.Nimda.Amm.exe?

Il nome del file e un depistaggio: non e il worm Nimda. Usando i frammenti leggibili posso dire con certezza di cosa si tratta. Il manifest XML incorporato dichiara:

```
<assemblyIdentity
  version="5.1.0.0"
  processorArchitecture="amd64"
  name="Microsoft.Windows.FileSystem.CMD"
  type="win32" />
<description>Windows Command Processor</description>
```

Il nome interno Microsoft.Windows.FileSystem.CMD e la descrizione windows Command Processor identificano l'eseguibile in modo inequivocabile: si tratta di **cmd.exe**, l'interprete dei comandi di Windows a 64 bit. La conferma arriva anche dalle stringhe presenti piu avanti nel flusso (Cmd:, Type:, Args:, Unknown type:) e dagli import tipici di una shell, come CreateProcessAsUserW, ShellExecuteExW e WNetAddConnection2W, accanto a numerose API di gestione del filesystem (FindFirstFileExW, CreateFileW, DeleteFileW, MoveFileExW).



```
.....8.....L.....h.....X.....
...&.....H.....Z.....j.....x.....memset...memcpy...memcpy...setjm
p.0.7terminate@YAXXZ..._set_app_type..._fmode..._commode..._setusermatherr..._amsg_exit...1..._initterm..._cexit..._exit...R...
XcptFilter.S..._C_specific_handler...q..._getmainargs..._calloc..._free..._wcslwr..._qsort..._dup2..._dup..._close..._open_osfhandl
e..._swscanf..._ultoa..._pipe..._wcsncpy..._setmode..._exit..._h_iswxdigit...time..._srand..._wtol..._fflush..._wcsstr..._iswalp...
_wcsstoul..._errno..._printf..._rand..._o_iob..._3_fprintf..._wcsrchr..._realloc..._tolower..._setlocale..._wcsupr..._a_iswdigit..._y_wcsic
mp..._f_iswspace..._wcschr..._memmove..._fgets..._pclose..._ferror..._feof..._wpopen..._wcsnicmp...X..._vsnwprintf..._wcstol..._D..._get_osfh
andle..._0..._getch..._toupper..._wcsspn..._tell..._s_longjmp..._local_unwind...{RtlCaptureContext...RtlLookupFunctionEntry...RtlVirtu
alUnwind...K...RtlFreeHeap...*...NtFsControlFile...1...NtOpenThreadToken...NtClose...d...NtOpenProcessToken...NtQueryInformationToken...RtlDosP
athNameToNtPathName_U...9...RtlFindLeastSignificantBit...NtSetInformationProcess...NtQueryInformationProcess...RtlNtStatusToDosErro
r...GetTimeFormatW...GetTickCount...QueryPerformanceCounter...SetUnhandledExceptionFilter...Sleep...DelayLoadFailureHook...?...Loa
dLibraryExA...g...FreeLibrary...CreateHardLinkW...CreateSymbolicLinkW...GetVolumePathNameW...GetThreadLocale...ResumeThread...SetP
rocessAffinityMask...0...GetNumaNodeProcessorMaskEx...GetThreadGroupAffinity...9...FindFirstFileExW...GetDiskFreeSpaceExW...K...FindNextS
treamW...@...FindFirstStreamW...DeviceIoControl...CompareFileTime...RemoveDirectoryW...GetCurrentDirectoryW...GetExitCodeProcess...
WaitForSingleObject...TerminateProcess...X...SetCurrentDirectoryW...t...SetFileTime...DeleteFileW...^...SetEndOfFile...k...SetFileAttribu
tesW...u...CopyFileW...CreateDirectoryW...Q...SetConsoleTextAttribute...+...FillConsoleOutputAttribute...&...ScrollConsoleScreenBufferW...m...GetACP
...c...FormatMessageW...\.FlushFileBuffers...DuplicateHandle...HeapSize...HeapReAlloc...VirtualAlloc...VirtualFree...HeapSetInfor
mation...GetCurrentThreadId...OpenThread...GetFileAttributesExW...GetDriveTypeW...GetVersion...LeaveCriticalSection...EnterC
riticalSection...GetModuleFileNameW...GetWindowsDirectoryW...8...SetConsoleCtrlHandler...InitializeCriticalSection...ExpandEnviron
mentStringsW...D...CancelSynchronousIo...GetVolumeInformationW...GlobalFree...GlobalAlloc...q...SetFilePointerEx...1...WriteFile...(.SearchPa
thW...J...LocalFree...S...SetConsoleTitleW...a...MoveFileExW...d...MoveFileW...QueryFullProcessImageNameW...ReadProcessMemory...A...LoadLibraryW...
RegSetValueExW...RegCreateKeyExW...UnhandledExceptionFilter...GetCurrentProcess...~...GetSystemTimeAsFileTime...VirtualQuery...\.Cm
dBatNotification...w...GetCPInfo...GetConsoleOutputCP...SetThreadLocale...I...GetProcAddress...GetModuleHandleW...R...CloseHandle...GetLa
stError...p...SetFilePointer...GetFullPathNameW...>...FindFirstFileW...J...FindNextFileW...3...FindClose...CreateFileW...ReadFile...h...MultiByt
eToWideChar...GetFileSize...WideCharToMultiByte...U...lstrcpw...W...lstrcpw...i...GetStdHandle...[...FlushConsoleInputBuffer...HeapAlloc...N...G
etProcessHeap...HeapFree...GetConsoleScreenBufferInfo...ReadConsoleW...<...SetConsoleCursorPosition...FillConsoleOutputCharacter
W...0...WriteConsoleW...GetFileType...GetUserDefaultLCID...GetLocaleInfoW...SetLocalTime...GetSystemTime...SystemTimeToFileTi
me...).FileTimeToLocalFileTime...*.FileTimeToSystemTime...GetDateFormatW...RegDeleteValueW...GetLocalTime...GetConsoleMode...H...Set
ConsoleMode...GetEnvironmentVariableW...GetCommandLineW...GetNumaHighestNodeNumber...GetEnvironmentStringsW...f...FreeEnvironmentS
tringsW...b...SetEnvironmentVariableW...SetEnvironmentStringsW...GetConsoleTitleW...GetVDMCurrentDirectories...c...SetErrorMode...Ini
tializeProcThreadAttributeList...UpdateProcThreadAttribute...DeleteProcThreadAttributeList...h...GetStartupInfoW...CreateProcessW...
GetConsoleWindow...GetFileAttributesW...j...NeedCurrentDirectoryForExePathW...v...GetBinaryTypeW...RegOpenKeyExW...RegCloseKey...|.SetLa
stError...RegQueryValueExW...RegDeleteKeyExW...GetCurrentProcessId...BrandingFormatString...d...d...T...p...d...T...4...
.r...p...d...T...4
```

Figura 4: il manifest XML incorporato nel flusso. I campi name="Microsoft.Windows.FileSystem.CMD" e Windows Command Processor rivelano la vera identita del file.





Figura 5: tabella degli import nel flusso. Le numerose API di filesystem e di gestione processi sono coerenti con un interprete dei comandi.

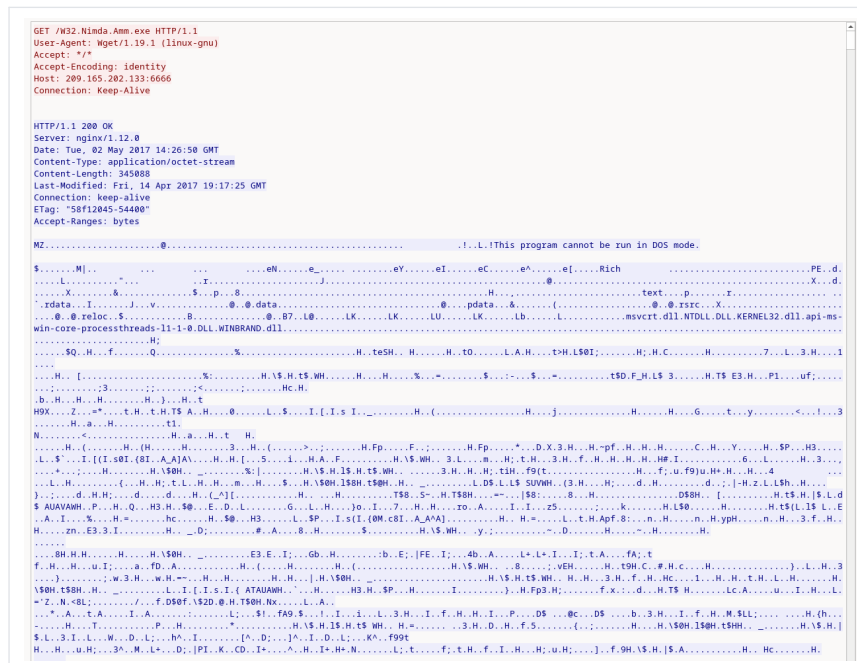


Figura 6: stringhe wide-char (Cmd:, Type:, Args:, Unknown type:) e librerie importate (ADVAPI32, SHELL32, KERNEL32, ntdll, msvcrt), ulteriore conferma che il binario è cmd.exe.

### Cosa significa dal punto di vista difensivo

Un attaccante ha rinominato un binario legittimo di sistema con un nome che richiama un malware famoso. Il nome di un file non dice nulla sul suo contenuto reale; solo l'analisi dei byte e degli attributi interni permette di stabilire la vera natura di un artefatto.

## 04

## Parte 2: Estrarre il file scaricato dal PCAP

Poiché la cattura contiene tutti i pacchetti del traffico, contiene anche tutti i byte del file scaricato. Wireshark può quindi ricostruire e salvare il file originale.

#### 4.1 Export Objects HTTP

Con il pacchetto della richiesta GET selezionato, ho aperto dal menu il percorso **File > Export Objects > HTTP**. Questa funzione analizza la cattura e ricostruisce tutti gli oggetti trasferiti via HTTP.

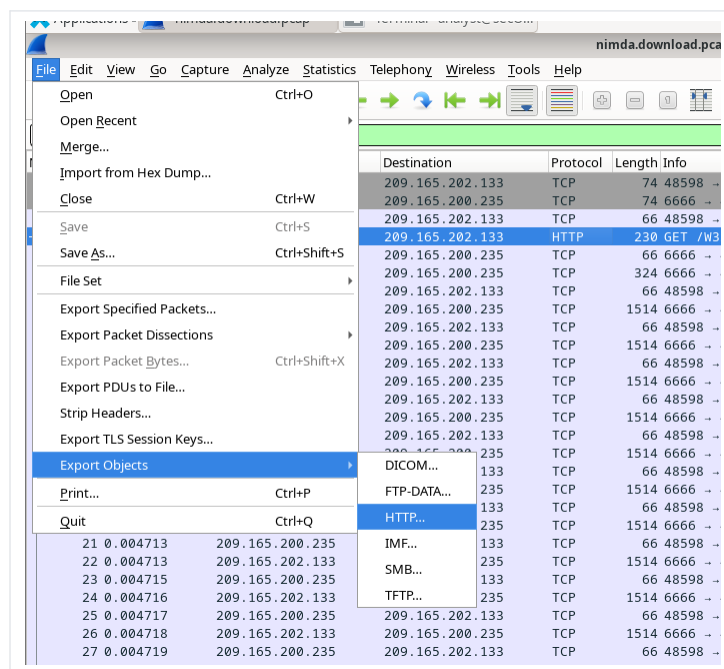


Figura 7: menu **File > Export Objects > HTTP** di Wireshark.

#### 4.2 Perché W32.Nimda.Amm.exe è l'unico file nella cattura

Nella finestra HTTP object list compare un solo oggetto:

| Pacchetto | Hostname             | Content Type             | Size   | Filename          |
|-----------|----------------------|--------------------------|--------|-------------------|
| 309       | 209.165.202.133:6666 | application/octet-stream | 345 kB | W32.Nimda.Amm.exe |

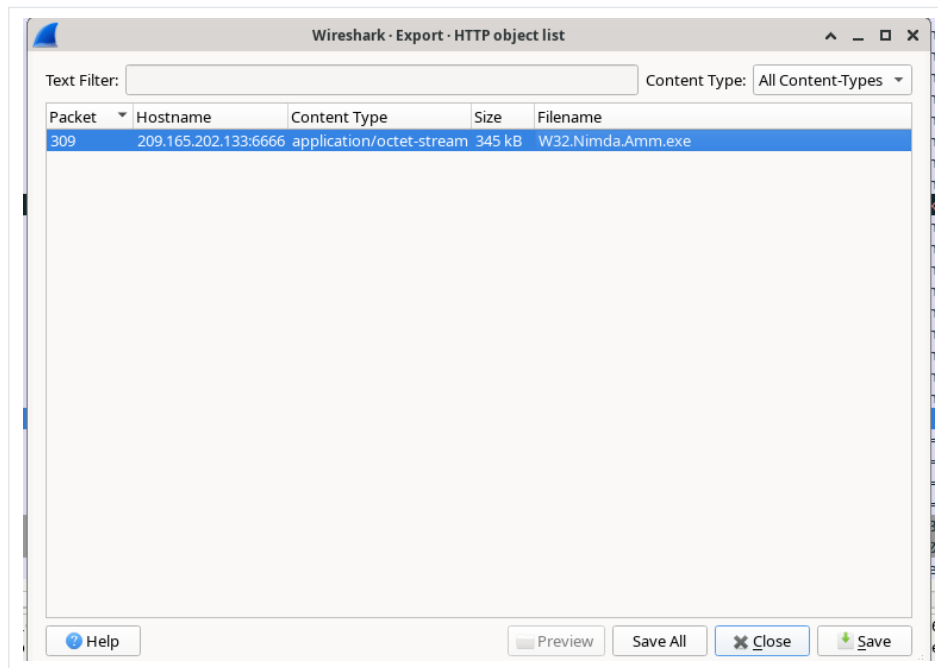


Figura 8: finestra HTTP object list con un unico oggetto, W32.Nimda.Amm.exe (345 kB), proveniente dal pacchetto 309.

E' l'unico file perche la cattura e stata realizzata per registrare esattamente quella singola transazione, cioe il download di quel preciso eseguibile. Il PCAP contiene una sola sessione HTTP con un solo oggetto trasferito, quindi Wireshark trova e ricostruisce solo quel file. Il pacchetto 309 e lo stesso frame che la richiesta GET indicava come Response in frame 309.

#### • 4.3 Salvataggio del file

Ho selezionato l'oggetto e usato save As, salvandolo nella cartella analyst (la home dell'utente). Sono poi tornato al terminale per verificare il salvataggio.

```
[analyst@secOps ~]$ cd /home/analyst
[analyst@secOps ~]$ ls -la
...
-rw-r--r-- 1 analyst analyst 345088 Jun 22 08:23 W32.Nimda.Amm.exe
```

**Il file e stato salvato?** Si. Il file W32.Nimda.Amm.exe e presente nella home con una dimensione di 345088 byte, identica al Content-Length annunciato dal server nella risposta HTTP. La corrispondenza esatta della dimensione mi conferma che la ricostruzione del file dal flusso TCP e avvenuta senza perdite.

```
[analyst@secOps ~]$ ls
analyst
[analyst@secOps ~]$ cd analyst/
[analyst@secOps ~]$ ls -la
total 524
drwx----- 16 analyst analyst 4896 Jun 22 08:25 .
drwxr-xr-x  3 root    root    4896 Mar 26 2018 ..
-rw-----  1 analyst analyst 453 Jun 22 08:06 .bash_history
-rw-r--r--  1 analyst analyst 21 Feb  7 2018 .bash_logout
-rw-r--r--  1 analyst analyst 57 Feb  7 2018 .bash_profile
-rw-r--r--  1 analyst analyst 97 Mar 20 2018 .bashrc
-rw-r--r--  1 analyst analyst 141 Feb  7 2018 .bashrc_stock
drwxr-xr-x  8 analyst analyst 4896 Jun 16 08:27 cache
-rw-r--r--  1 root    root    6665 Jun 16 08:29 capture.pcap
drwxr-xr-x 12 analyst analyst 4896 Jun 22 08:23 config
drwxr-xr-x  2 analyst analyst 4896 Jun 22 08:25 Desktop
-rw-r--r--  1 analyst analyst 23 Mar 23 2018 .dmrc
drwxr-xr-x  3 analyst analyst 4896 Jun 18 2025 Downloads
drwx-----  4 analyst analyst 4896 Jun 19 2025 gnupg
-rw-----  1 analyst analyst 2528 Jun 17 2025 .ICEauthority
drwxr-xr-x  2 analyst analyst 4896 Mar 24 2018 .idlerc
drwxr-xr-x  9 analyst analyst 4896 Jun 18 2025 lab.support.files
-rw-----  1 analyst analyst 153 Jun 19 05:09 .lesshst
drwxr-xr-x  3 analyst analyst 4896 Mar 22 2018 local
drwx-----  5 analyst analyst 4896 Mar 24 2018 mozilla
drwxr-xr-x  3 analyst analyst 4896 Jun 18 2025 scripts
drwxr-xr-x  2 analyst analyst 4896 Mar 21 2018 second_drive
drwx-----  2 analyst analyst 4896 Apr  2 2018 .ssh
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-clipboard-tty7-control.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-clipboard-tty7-service.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-display-svga-x11-tty7-control.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-display-svga-x11-tty7-service.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-draganddrop-tty7-control.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-draganddrop-tty7-service.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-hostversion-tty7-control.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-seamless-tty7-control.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-seamless-tty7-service.pid
-rw-r-----  1 analyst analyst 4 Jun 22 07:53 .vboxclient-vmsvga-session-tty7-control.pid
drwxr-xr-x  3 analyst analyst 4896 Mar 26 2018 .vim
-rw-----  1 analyst analyst 13912 Jul 19 2018 viminfo
-rw-r--r--  1 analyst analyst 345088 Jun 22 08:23 W32.Nimda.Amm.exe ←
-rw-----  1 analyst analyst 51 Jun 22 07:53 .xauthority
-rw-r--r--  1 analyst analyst 16 Mar 22 2018 .xinitrc
-rw-r--r--  1 analyst analyst 16 Mar 22 2018 .xinitrc
-rw-----  1 analyst analyst 633 Jun 22 07:53 .xsession-errors
-rw-----  1 analyst analyst 635 Jun 19 05:02 .xsession-errors.old
-rwxr-xr-x  1 analyst analyst 90 Jun 17 2025 .xsessionrc
drwxr-xr-x  5 analyst analyst 4896 Jun 18 2025 yay
[analyst@secOps ~]$
```

Figura 9: `ls -la` in `/home/analyst`. Il file `W32.Nimda.Amm.exe` da 345088 byte e stato salvato correttamente.

#### 4.4 Identificazione del tipo di file

Il comando `file` analizza i primi byte di un file (i magic number) e ne deduce il tipo reale, indipendentemente dall'estensione. L'ho usato sul file estratto.

```
[analyst@secOps ~]$ file W32.Nimda.Amm.exe
W32.Nimda.Amm.exe: PE32+ executable for MS Windows 6.01 (console), x86-64, 6 sections
```

##### Cosa significa il risultato

PE32+ indica un eseguibile Windows a 64 bit, x86-64 ne conferma l'architettura, console segnala un'applicazione a riga di comando e 6 sections corrisponde esattamente alle sei sezioni (.text, .rdata, .data, .pdata, .rsrc, .reloc) che avevo già individuato nel Follow TCP Stream. Tutto coerente con l'identificazione fatta nella Parte 1: il file è un eseguibile Windows a 64 bit, di fatto `cmd.exe`.

```
[analyst@secOps ~]$ file W32.Nimda.Amm.exe
W32.Nimda.Amm.exe: PE32+ executable for MS Windows 6.01 (console), x86-64, 6 sections
[analyst@secOps ~]$
```

Figura 10: output del comando `file` sul campione estratto, classificato come PE32+ executable x86-64 a sei sezioni.

#### 4.5 Quale sarebbe un probabile passo successivo per un analista di sicurezza?

Una volta isolato e identificato l'artefatto, il passo successivo è l'analisi del malware vera e propria, da svolgere sempre in un ambiente controllato e isolato dalla rete di produzione. Un percorso ragionevole è:

1. **Calcolo degli hash** del file (MD5, SHA-1, SHA-256) per ottenere un identificativo univoco e ricercabile.
2. **Confronto con la threat intelligence**: verificare gli hash su servizi come VirusTotal o database interni di IoC, per capire se l'artefatto è già noto.

3. **Analisi statica:** esaminare header PE, tabella degli import, sezioni e stringhe senza eseguire il file, per ipotizzarne il comportamento.

4. **Analisi dinamica in sandbox:** eseguire il campione in una sandbox isolata (ad esempio ANY.RUN, tria.ge, Cuckoo o una VM dedicata con FakeNet-NG e Wireshark) per osservare cosa fa davvero: processi creati, modifiche al filesystem e al registro, connessioni di rete e canali C2.

Il passo immediato che ha piu senso, come ho ipotizzato, e proprio l'esecuzione in sandbox controllata: e il modo piu sicuro per osservare il comportamento reale del file senza rischiare il sistema, e affianca l'analisi statica gia impostata con `file` e con la lettura del flusso. In questo caso specifico l'analisi ha rivelato che dietro il nome allarmante c'e un binario legittimo di sistema, ma in uno scenario reale questa e la fase in cui si determina se l'artefatto e davvero malevolo e quali indicatori di compromissione produce.

## 05

## Riepilogo dei comandi

| Comando                                          | Scopo                                            |
|--------------------------------------------------|--------------------------------------------------|
| <code>cd lab.support.files/pcaps/</code>         | spostarsi nella cartella della cattura           |
| <code>ls -la</code>                              | elencare i file con dettagli e dimensioni        |
| <code>wireshark nimda.download.pcap &amp;</code> | aprire il PCAP in Wireshark in background        |
| File > Export Objects > HTTP                     | ricostruire gli oggetti HTTP dalla cattura       |
| <code>cd /home/analyst</code>                    | tornare nella home per verificare il salvataggio |
| <code>file W32.Nimda.Amm.exe</code>              | identificare il tipo reale del file estratto     |

## 06

## Conclusioni

In questo laboratorio ho seguito l'intero ciclo di analisi di un download catturato in rete, dalla connessione TCP fino al recupero e all'identificazione dell'eseguibile. Ho verificato l'handshake a tre vie, individuato la richiesta HTTP GET prodotta da `wget`, ricostruito il flusso completo con Follow TCP Stream e riconosciuto la struttura interna di un file PE di Windows direttamente dai byte trasmessi.

Il risultato più significativo è che il nome `w32.Nimda.Amm.exe` è ingannevole: il manifest incorporato e gli import del file dimostrano che si tratta in realtà di `cmd.exe`, il Windows Command Processor a 64 bit. Ho poi estratto il file con Export Objects, ne ho confermato il salvataggio confrontando la dimensione con il Content-Length della risposta HTTP, e ne ho verificato la natura con il comando `file`, che lo ha classificato come PE32+ executable x86-64 a sei sezioni.

La lezione operativa è duplice: una cattura di rete non contiene solo metadati ma i dati veri e propri, recuperabili integralmente; e il nome di un file non ha alcun valore probatorio sulla sua natura, che va sempre stabilita analizzandone il contenuto.